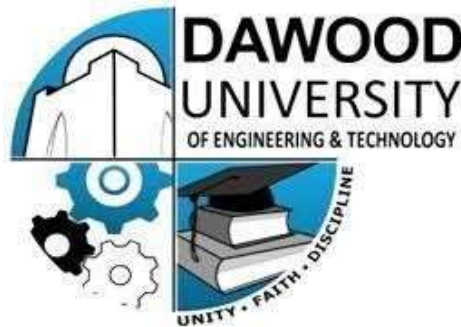


Machine Learning

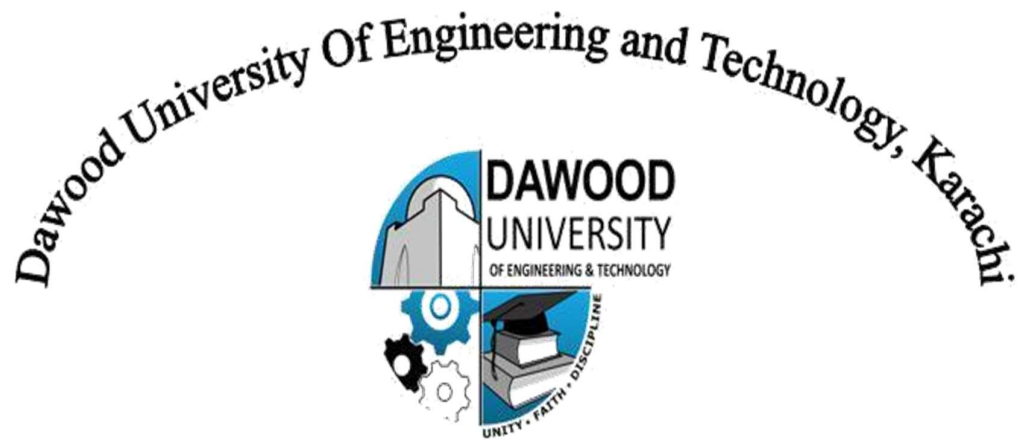
(Practical Manual)



**5th Semester, 3rd Year
BATCH -2024**

BS ARTIFICIAL INTELLIGENCE

DAWOOD UNIVERSITY OF ENGINEERING & TECHNOLOGY, KARACHI



CERTIFICATE

This is to certify that Mr. Abdul Hayy Khan with Roll # 24F-AI-051 of Batch 2024 has successfully completed all the labs prescribed for the course “Artificial Intelligence”.

Engr. Hamza Farooqui
Lecturer
Department of AI

LAB RULES AND OPERATING PROCEDURES

Before starting a lab, you must read the following instructions. Failure to conform to any of the below Instructions may result in not being allowed to participate in the laboratory experiment.

Respect the Lab Rules:

Respect the specific guidelines provided by the lab supervisor or instructor.

Quiet Environment:

Keep noise levels to a minimum to create a conducive learning environment for everyone.

No Food or Drinks:

Do not bring food or drinks into the computer lab to prevent spills and maintain cleanliness.

Log in with Your Own Credentials:

Use only your assigned login credentials, and do not attempt to access another student's account.

Log Out Properly:

Always log out of the computer and any applications or platforms you use before leaving the computer.

Respect Equipment:

Treat computer equipment and peripherals with care. Report any malfunctioning equipment to lab staff.

No Unauthorized Software Installation:

Do not install or attempt to install any software on the lab computers without permission from lab staff or instructors.

No Tampering with Hardware:

Do not tamper with computer hardware or cables. Report any issues to lab staff.

Internet Usage:

Use the internet for educational purposes only. Avoid accessing inappropriate or non-educational websites.

Be Mindful of Time:

Be aware of the lab's opening and closing hours. Finish your work and leave the lab on time.

Personal Belongings:

Keep personal belongings secure. Do not leave valuables unattended.

Emergency Procedures:

Familiarize yourself with emergency procedures, including the location of exits and emergency contacts.

I have read and understand these rules and procedures. I agree to abide by these rules and procedures at all times while using these facilities. I understand that failure to follow these rules and procedures will result in my immediate dismissal from the laboratory and additional disciplinary action may be taken.

Student's Signature

COURSE INFORMATION SHEET (For Lab Based Course)

Title of Course: Machine Learning (Practical)

Course Code: AI-3202

Effectiveness: Batch 2023-F and onwards

Credit Hours: 01 CH (Practical)

Instructor Name: Engr. Hamza Farooqui

Email and Contact Information: hamza.farooqui@duet.edu.pk

Lab Assessment: Lab performance is evaluated based on comprehensive rubric for each experiment, marked out of 20 and Complex Computing Activity based on 10 marks. The key criteria include:

- **Understanding of Concept (CLO-1):** Clarity and depth of ML algorithm.
- **Code Implementation (CLO-1):** Structure, correctness, and robustness of the Python code.
- **Use of ML Libraries & Features (CLO-2):** Appropriate and efficient use of ML tools for data analysis, training, and evaluation.
- **Results and Report (CLO-2):** Accuracy of the model results and clarity in performance evaluation.

Aim:

The aim of this lab is to equip students with practical skills and knowledge to apply machine learning algorithms for data-driven decision-making through hands-on implementation and real-world problem scenarios.

Objectives:

- The objective of this lab is to develop proficiency in data preprocessing, model training, and performance evaluation of machine learning algorithms.
- To build an understanding of how to select appropriate ML models for given datasets and optimize them for better accuracy and generalization.

Course Learning Outcomes (CLOs):

Upon successful completion of this course, students will be able to:

Mapping of CLOs and GAs			
Sr. No	Course Learning Outcomes	GAs	Knowledge Level
CLO-1	Practice fundamental machine learning algorithms to solve real-world problems using Python libraries.	GA-3	P-3
CLO-2	Demonstrate the ability to evaluate the performance of machine learning models using performance metrics and optimize models through techniques like hyperparameter tuning.	GA-5	P-4
GA= Graduate Attribute, C = Cognitive Domain, P = Psychomotor Domain, A= Affective Domain			

Complex Computing Activity (CCA) Details	Included: Yes Nature and details of Complex Computing Activity (CCA): • Conducted as a subject project in groups of 2–4 students. • The project involves designing and implementing an advanced machine learning application that integrates multiple ML techniques (e.g., supervised and unsupervised learning, model optimization, and data visualization). • Students are required to collect, preprocess, and analyze data, build and evaluate models, and present insights or real-time results tools and simulation environments. • Range of Computing Activity Involved 1 and 2 • Assessment through Report and Viva.
---	--

Assessment Breakdown and CLO mapping:

Assessment Tool	Marks	Mapped CLOs	CLO-1 Marks	CLO-2 Marks	CLO-3 Marks
Lab work/Report	20	CLO-1, CLO-2	10	10	–
CCA Project	10	CLO-1, CLO-2	5	5	–
Lab Exam	10	CLO-1, CLO-2	5	5	–
Viva	10	CLO-2	–	–	10
Total	50	–	20	20	10

TABLE OF CONTENTS

S. No.	Title of Experiment
1	Data Preprocessing: Cleaning, Encoding, and Feature Scaling
2	Simple and Multiple Linear Regression
3	Logistic Regression: Binary and Multiclass Classification
4	Decision Tree Classifier
5	Random Forest Classifier
6	Support Vector Machine (SVM) Classifier
7	Open Ended Lab
8	K-Nearest Neighbors (KNN) Classifier
9	Naïve Bayes Classifier
10	K-Means Clustering
11	Hierarchical Agglomerative Clustering
12	ML × AR — Real-Time Object Classification Overlay
13	ML × AR — Gesture Recognition
14	Complex Computing Activity

LAB # 01

DATA PREPROCESSING

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0-1 marks)	Marks (out of 20)
Understanding of Concept (4 marks)	Demonstrates clear and in-depth understanding of theory; strongly relates it well to lab objectives.	Basic understanding of theory; provides minimal or partial connection to lab objectives.	Little or no understanding of concept; unclear or incorrect relevance to lab topic.	
Code Implementation (6 marks)	Code is well-structured, correct, error-free, and reflects the theoretical concepts; handles edge cases effectively.	Code works with minor errors or inefficiencies; shows partial understanding of the concept.	Code is incorrect, incomplete, or does not align with lab goals.	
Use of Programming Features/Tools (4 marks)	Efficiently applies relevant Python libraries, data processing methods, and ML techniques (e.g., NumPy, Pandas, scikit-learn, Matplotlib) to achieve the desired outcomes.	Uses standard/basic functions and logic; limited or partial use of available features/tools.	Minimal or incorrect use of tools; relies on trivial constructs or hard-coded approaches.	
Results and Report (6 marks)	Produces accurate and well-presented results (visualizations, metrics, comparisons) with clear interpretation and insights.	Results are partially correct or lack clarity in interpretation; report is adequate but lacks depth, formatting, or coherence.	Results are missing, incorrect, or poorly explained.	

LAB # 01

DATA PREPROCESSING

OBJECTIVE

To understand and implement Data preprocessing in preparing real-world datasets for machine learning.

THEORY

Before training any machine learning model, it is crucial to prepare and clean the dataset so that algorithms can interpret it correctly. Data preprocessing improves the accuracy, speed, and reliability of the model.

1. Dataset Cleaning: -

Raw datasets often contain missing, duplicate, or inconsistent values, which can mislead a model. The Titanic dataset, for instance, has missing values in columns like *Age* and *Embarked*.

Handling Missing Data: -

- **Detection:** Identify missing values using methods like `isnull()` or `info()`.
- **Imputation:** Replace missing numeric values (like *Age*) with statistical measures such as:
 - Mean → useful for normally distributed data
 - Median → less sensitive to outliers
 - Mode → for categorical variables
- **Dropping:** If a feature has too many missing values or is irrelevant, it may be dropped entirely.

Example:

- Fill missing *Age* values with the mean age.
- Drop rows where *Embarked* is missing.

Proper handling of missing data ensures that the dataset remains representative and the model learns from valid patterns.

2. Encoding Categorical Data: -

Machine learning models work with numerical data, so categorical (text) data must be converted into numbers.

a. Label Encoding

Converts categories into integers.

Example:

Sex → Male = 0, Female = 1

b. One-Hot Encoding

Creates separate binary columns for each category.

Example:

Embarked → C, Q, S

becomes:

Embarked_C, Embarked_Q, Embarked_S (values are 0 or 1)

One-Hot Encoding prevents the model from assuming an order or hierarchy among categories.

3. Feature Scaling: -

Different features can have values in different ranges (e.g., *Fare* may range from 0–500, while *Age* ranges from 0–80).

Algorithms that depend on distance metrics (e.g., Logistic Regression, SVM, KNN) perform poorly if features are not scaled.

Standardization (Z-score normalization):

Rescales features so that they have:

- Mean = 0
- Standard Deviation = 1

This ensures all features contribute equally to the model's learning process.

4. Splitting the Dataset: -

To evaluate model performance, the dataset is divided into:

- **Training set (80%)** – used to train the model
- **Testing set (20%)** – used to evaluate accuracy on unseen data

This prevents overfitting and helps assess how well the model generalizes.

LAB TASKS

Task 1 – Dataset Cleaning

- Load the Titanic dataset (train.csv).
- Display the first 10 rows.
- Check for missing values in each column.
- Fill missing values in the "Age" column with the mean age.
- Drop rows where the "Embarked" column is missing.

Task 2 – Encoding Categorical Data

- Convert the "Sex" column into numeric (0 = Male, 1 = Female).
- Apply One-Hot Encoding on the "Embarked" column.

Task 3 – Feature Scaling & Splitting

- Select features: Age, Fare, Sex, Pclass.
- Apply StandardScaler to normalize them.
- Split data into 80% training and 20% testing.

LAB OUTCOMES

- Understand the importance of data preprocessing in preparing real-world datasets for machine learning.

- Identify and handle missing values using imputation and row removal techniques.
- Encode categorical features (e.g., gender, embarkation port) into numeric form suitable for algorithms.
- Normalize numerical features using feature scaling to ensure equal contribution of all variables.
- Split datasets into training and testing sets for model evaluation and validation.

Lab # 01: Data Preprocessing

Cleaning, Encoding, and Feature Scaling

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Task 1: Dataset Cleaning

1. Load the Titanic dataset (train.csv).
2. Display the first 10 rows.
3. Check for missing values in each column.
4. Fill missing values in the "Age" column with the mean age.
5. Drop rows where the "Embarked" column is missing.

```
import pandas as pd

# 1. Load the Titanic dataset (train.csv)
df = pd.read_csv('datasets/train.csv')

# 2. Display the first 10 rows
print("First 10 rows:")
display(df.head(10))
```

First 10 rows:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
5	6	0	3	
6	7	0	1	
7	8	0	3	
8	9	1	3	
9	10	1	2	

SibSp	\	Name	Sex	Age
0		Braund, Mr. Owen Harris	male	22.0
1				
1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
1				
2		Heikkinen, Miss. Laina	female	26.0
0				
3		Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
1				

```

4         Allen, Mr. William Henry    male  35.0
0
5         Moran, Mr. James           male   NaN
0
6         McCarthy, Mr. Timothy J    male  54.0
0
7         Palsson, Master. Gosta Leonard  male   2.0
3
8  Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)  female  27.0
0
9         Nasser, Mrs. Nicholas (Adele Achem)  female  14.0
1

```

```

      Parch      Ticket    Fare Cabin Embarked
0         0    A/5 21171    7.2500   NaN      S
1         0      PC 17599   71.2833   C85      C
2         0  STON/O2. 3101282    7.9250   NaN      S
3         0      113803   53.1000  C123      S
4         0      373450    8.0500   NaN      S
5         0      330877    8.4583   NaN      Q
6         0      17463   51.8625   E46      S
7         1      349909   21.0750   NaN      S
8         2      347742   11.1333   NaN      S
9         0      237736   30.0708   NaN      C

```

```

# 3. Check for missing values in each column
print("\nMissing values before cleaning:")
print(df.isnull().sum())

```

```

Missing values before cleaning:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin        687
Embarked       2
dtype: int64

```

```

# 4. Fill missing values in the "Age" column with the mean age
df['Age'] = df['Age'].fillna(df['Age'].mean())

```

```

# 5. Drop rows where the "Embarked" column is missing

```

```
df = df.dropna(subset=['Embarked'])

# Verify the changes
print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         0
dtype: int64
```

Task 2: Encoding Categorical Data

1. Convert the "Sex" column into numeric (0 = Male, 1 = Female).
2. Apply One-Hot Encoding on the "Embarked" column.

```
# 1. Convert the "Sex" column into numeric (0 = Male, 1 = Female).
# Using map to assign the numeric values based on the requirements
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

```
# 2. Apply One-Hot Encoding on the "Embarked" column.
# pd.get_dummies automatically applies one-hot encoding
# We set drop_first=True to avoid the dummy variable trap
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
```

```
# Display the first few rows to verify the transformations
display(df.head())
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Parch	\	Name	Sex	Age	SibSp
0			Braund, Mr. Owen Harris	0	22.0	1
0						
1			Cumings, Mrs. John Bradley (Florence Briggs Th...	1	38.0	1

0						
2			Heikkinen, Miss. Laina	1	26.0	0
0						
3			Futrelle, Mrs. Jacques Heath (Lily May Peel)	1	35.0	1
0						
4			Allen, Mr. William Henry	0	35.0	0
0						
	Ticket	Fare	Cabin	Embarked_Q	Embarked_S	
0	A/5 21171	7.2500	NaN	False	True	
1	PC 17599	71.2833	C85	False	False	
2	STON/O2. 3101282	7.9250	NaN	False	True	
3	113803	53.1000	C123	False	True	
4	373450	8.0500	NaN	False	True	

Task 3: Feature Scaling & Splitting

1. Select features: Age, Fare, Sex, Pclass.
2. Apply StandardScaler to normalize them.
3. Split data into 80% training and 20% testing.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 1. Select features: Age, Fare, Sex, Pclass
features = ['Age', 'Fare', 'Sex', 'Pclass']
X = df[features]

# We also need to define the target variable 'y' to split the data properly
y = df['Survived']

# 2. Apply StandardScaler to normalize them
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Split data into 80% training and 20% testing
# test_size=0.2 represents the 20% testing data. random_state ensures reproducibility.
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

# Verify the shapes of the splits
print(f"Training data shape: X_train: {X_train.shape}, y_train: {y_train.shape}")
print(f"Testing data shape: X_test: {X_test.shape}, y_test: {y_test.shape}")

Training data shape: X_train: (711, 4), y_train: (711,)
Testing data shape: X_test: (178, 4), y_test: (178,)
```

LAB # 02

SIMPLE AND MULTIPLE LINEAR REGRESSION

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0-1 marks)	Marks (out of 20)
Understanding of Concept (4 marks)	Demonstrates clear and in-depth understanding of theory; strongly relates it well to lab objectives.	Basic understanding of theory; provides minimal or partial connection to lab objectives.	Little or no understanding of concept; unclear or incorrect relevance to lab topic.	
Code Implementation (6 marks)	Code is well-structured, correct, error-free, and reflects the theoretical concepts; handles edge cases effectively.	Code works with minor errors or inefficiencies; shows partial understanding of the concept.	Code is incorrect, incomplete, or does not align with lab goals.	
Use of Programming Features/Tools (4 marks)	Efficiently applies relevant Python libraries, data processing methods, and ML techniques (e.g., NumPy, Pandas, scikit-learn, Matplotlib) to achieve the desired outcomes.	Uses standard/basic functions and logic; limited or partial use of available features/tools.	Minimal or incorrect use of tools; relies on trivial constructs or hard-coded approaches.	
Results and Report (6 marks)	Produces accurate and well-presented results (visualizations, metrics, comparisons) with clear interpretation and insights.	Results are partially correct or lack clarity in interpretation; report is adequate but lacks depth, formatting, or coherence.	Results are missing, incorrect, or poorly explained.	

LAB # 02

SIMPLE AND MULTIPLE LINEAR REGRESSION

OBJECTIVE

To understand and implement Simple Linear Regression and Multiple Linear Regression.

THEORY:

Practical Significance

Linear regression is one of the most fundamental and widely used machine learning algorithms. It is used for predicting numerical values based on one or more independent variables. This lab covers:

1. Simple Linear Regression: Predicting a target variable using a single independent variable.
2. Multiple Linear Regression: Extending linear regression to multiple features to improve predictions.

Minimum Theoretical Background

1. Simple Linear Regression

- Models the relationship between a dependent variable Y and a single independent variable X .

Equation:

$$Y = mX + b$$

where:

- Y = Dependent variable (target)
- X = Independent variable (feature)
- m = Slope of the regression line
- b = Intercept

2. Multiple Linear Regression

- Generalizes simple linear regression to multiple features:

$$Y = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$$

- b_0 is the intercept, and $b_1, b_2, \dots, b_n$ are the coefficients for features $X_1, X_2, \dots, X_n$.
- Used when multiple factors influence the dependent variable.

3. Model Evaluation

- Mean Squared Error (MSE)
- R-squared Score: Measures how well the model explains the variance in the data.

Mathematical Expression

1. Loss Function:

- The model minimizes the **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

- Where Y_i is the actual value and $\hat{Y}_i$ is the predicted value.

2. Gradient Descent (Optimization):

- Adjusts parameters to minimize the error:

$$b_j = b_j - \alpha \frac{\partial MSE}{\partial b_j}$$

- Where α is the learning rate.

LAB TASKS

Task 1: Simple Linear Regression:-

- **Load dataset:** Use the Diabetes dataset from `sklearn.datasets`. Select one feature (bmi) to predict the target (disease progression).
- **Perform Exploratory Data Analysis (EDA):**
 - Plot scatter plot of BMI vs. Disease Progression.
 - Check correlation.
- **Implement Simple Linear Regression** using `sklearn.linear_model.LinearRegression`:
 - Split data into training and testing sets.
 - Fit the model and predict disease progression.
 - Plot the regression line on the scatter plot.
- **Evaluate the model** using:
 - Mean Squared Error (MSE)
 - R^2 score

Does BMI alone explain most of the variation in disease progression? How does R^2 help explain this?

Task 2: Multivariate Linear Regression: -

- **Load dataset:** Use the same Diabetes dataset, but **include all 10 features** to predict disease progression.
- **Perform EDA:**

- Generate a correlation heatmap between features and the target.
 - Create pair plots for selected features vs. target.
- **Implement Multivariate Linear Regression:**
 - Use all independent variables to predict the target.
 - Fit and predict using the model.
- **Compare actual vs. predicted values** using:
 - Scatter plot (predicted vs. actual).
 - Residual plot.
- **Evaluate model performance** with:
 - MSE
 - RMSE
 - R^2 score

Which independent variable contributes the most to predicting disease progression?

Task 3: Experimentation: -

- Compare performance of simple vs. multivariate regression in terms of evaluation metrics.

LAB OUTCOMES

By completing this lab, students will:

- Implement Simple Linear Regression and Multiple Linear Regression.
- Apply data preprocessing to prepare datasets for regression models.
- Evaluate model performance using R-squared Score.

Lab # 02: Simple and Multiple Linear Regression

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Task 1: Simple Linear Regression

- Load dataset: Use the Diabetes dataset from sklearn.datasets. Select one feature (bmi) to predict the target (disease progression).
- Perform Exploratory Data Analysis (EDA):
 - Plot scatter plot of BMI vs. Disease Progression.
 - Check correlation.
- Implement Simple Linear Regression using sklearn.linear_model.LinearRegression:
 - Split data into training and testing sets.
 - Fit the model and predict disease progression.
 - Plot the regression line on the scatter plot.
- Evaluate the model using:
 - Mean Squared Error (MSE)
 - R^2 score

Does BMI alone explain most of the variation in disease progression? How does R^2 help explain this?

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

import matplotlib.pyplot as plt

# Load the Diabetes dataset
diabetes = load_diabetes(as_frame=True)
df = diabetes.frame

# Select BMI as the predictor and disease progression as the target
X_bmi = df[['bmi']]
y = df['target']

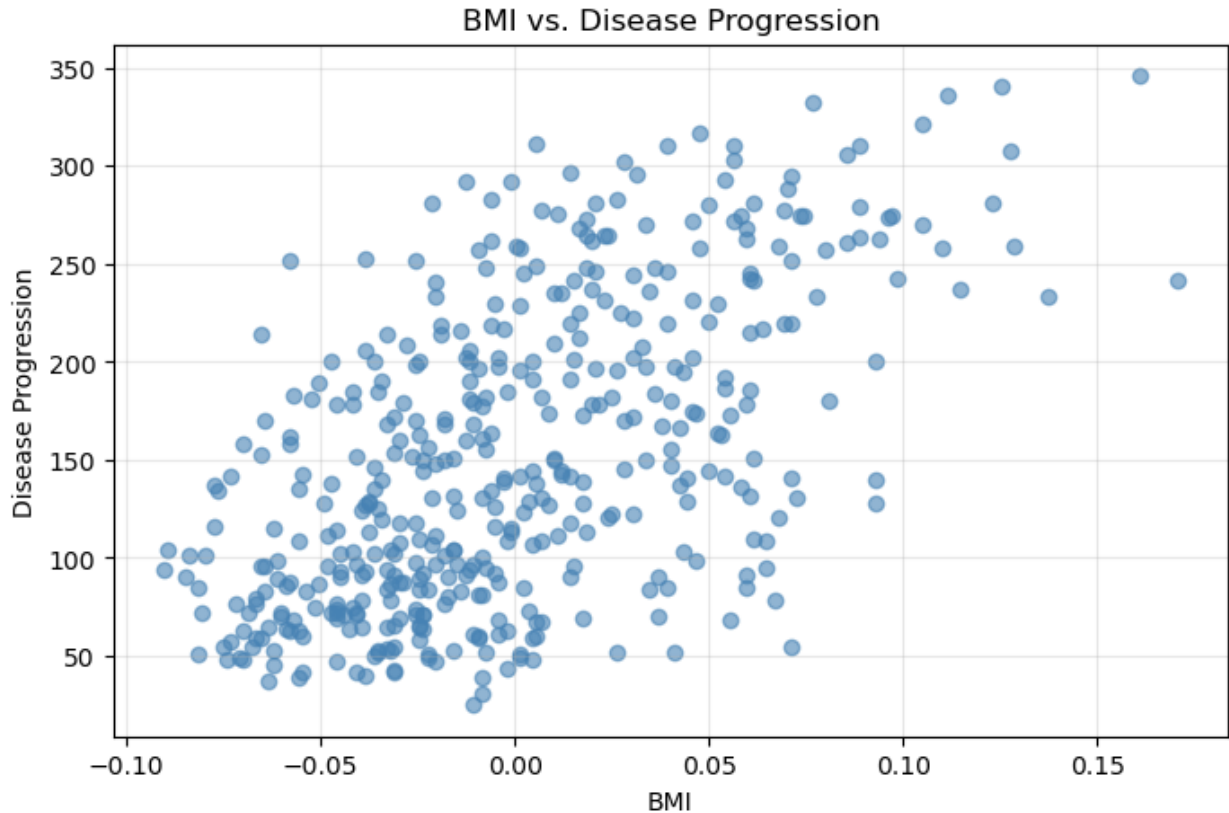
# Exploratory Data Analysis: correlation
correlation = X_bmi['bmi'].corr(y)
print(f"Correlation between BMI and disease progression:
{correlation:.4f}")

Correlation between BMI and disease progression: 0.5865
```

```

# Scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(X_bmi['bmi'], y, alpha=0.6, color='steelblue')
plt.title("BMI vs. Disease Progression")
plt.xlabel("BMI")
plt.ylabel("Disease Progression")
plt.grid(alpha=0.3)
plt.show()

```



```

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X_bmi, y, test_size=0.2, random_state=42
)

# Train the simple linear regression model
slr_model = LinearRegression()
slr_model.fit(X_train, y_train)

LinearRegression()

# Predictions
y_pred_slr = slr_model.predict(X_test)

```

```

# Evaluation metrics
mse_slr = mean_squared_error(y_test, y_pred_slr)
rmse_slr = np.sqrt(mse_slr)
r2_slr = r2_score(y_test, y_pred_slr)
print(f"Mean Squared Error (MSE): {mse_slr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_slr:.2f}")
print(f"R-squared Score (R²): {r2_slr:.4f}")

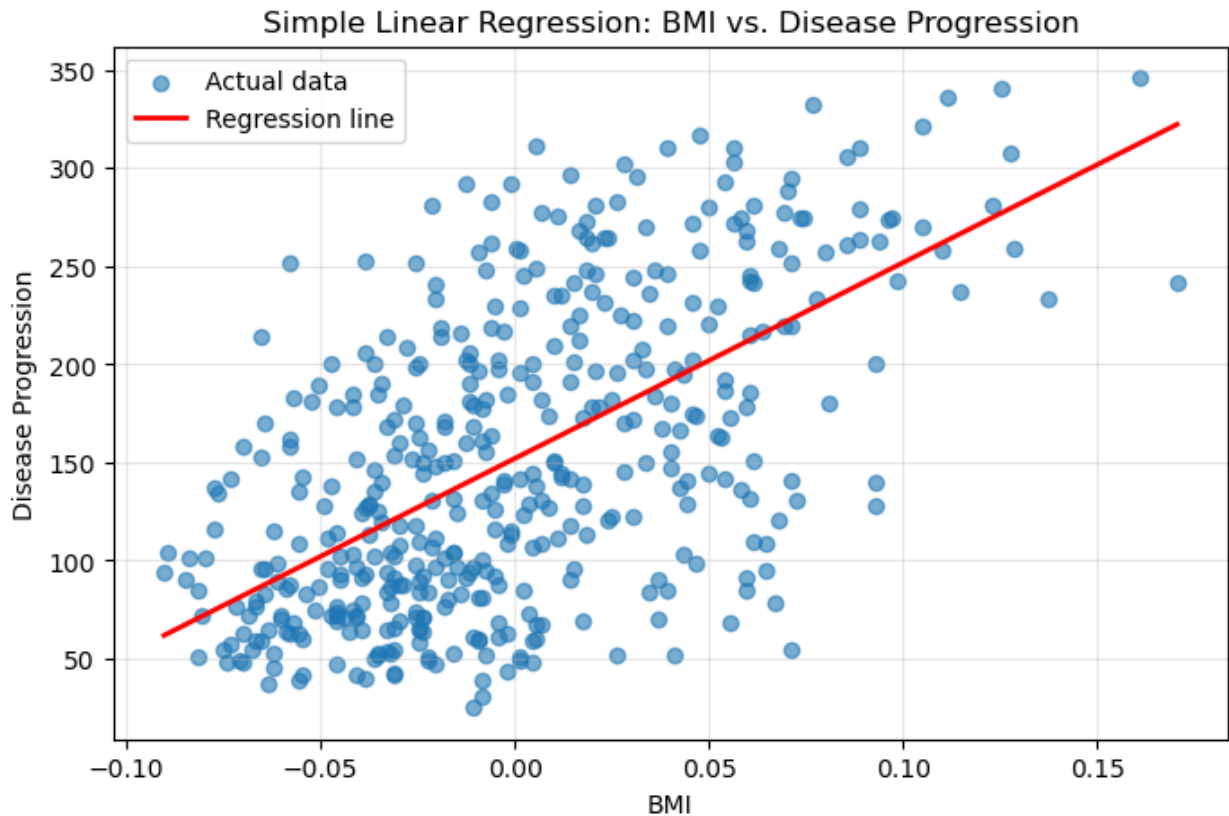
Mean Squared Error (MSE): 4061.83
Root Mean Squared Error (RMSE): 63.73
R-squared Score (R²): 0.2334

# Plot regression line
x_line = np.linspace(X_bmi['bmi'].min(), X_bmi['bmi'].max(),
100).reshape(-1, 1)
y_line = slr_model.predict(x_line)

plt.figure(figsize=(8, 5))
plt.scatter(X_bmi['bmi'], y, alpha=0.6, label="Actual data")
plt.plot(x_line, y_line, color='red', linewidth=2, label="Regression
line")
plt.title("Simple Linear Regression: BMI vs. Disease Progression")
plt.xlabel("BMI")
plt.ylabel("Disease Progression")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

c:\Users\USER\anaconda3\Lib\site-packages\sklearn\utils\
validation.py:2739: UserWarning: X does not have valid feature names,
but LinearRegression was fitted with feature names
  warnings.warn(

```



```
print(
    f"BMI alone explains approximately {r2_slr * 100:.2f}% of the\n"
    "variation "
    "in disease progression. The R2 score measures the proportion of\n"
    "target "
    "variation explained by the model, so BMI does not explain most of\n"
    "the variation."
)
```

BMI alone explains approximately 23.34% of the variation in disease progression. The R^2 score measures the proportion of target variation explained by the model, so BMI does not explain most of the variation.

Task 2: Multivariate Linear Regression

- Load dataset: Use the same Diabetes dataset, but include all 10 features to predict disease progression.
- Perform EDA:
 - Generate a correlation heatmap between features and the target.
 - Create pair plots for selected features vs. target.
- Implement Multivariate Linear Regression:
 - Use all independent variables to predict the target.
 - Fit and predict using the model.
- Compare actual vs. predicted values using:

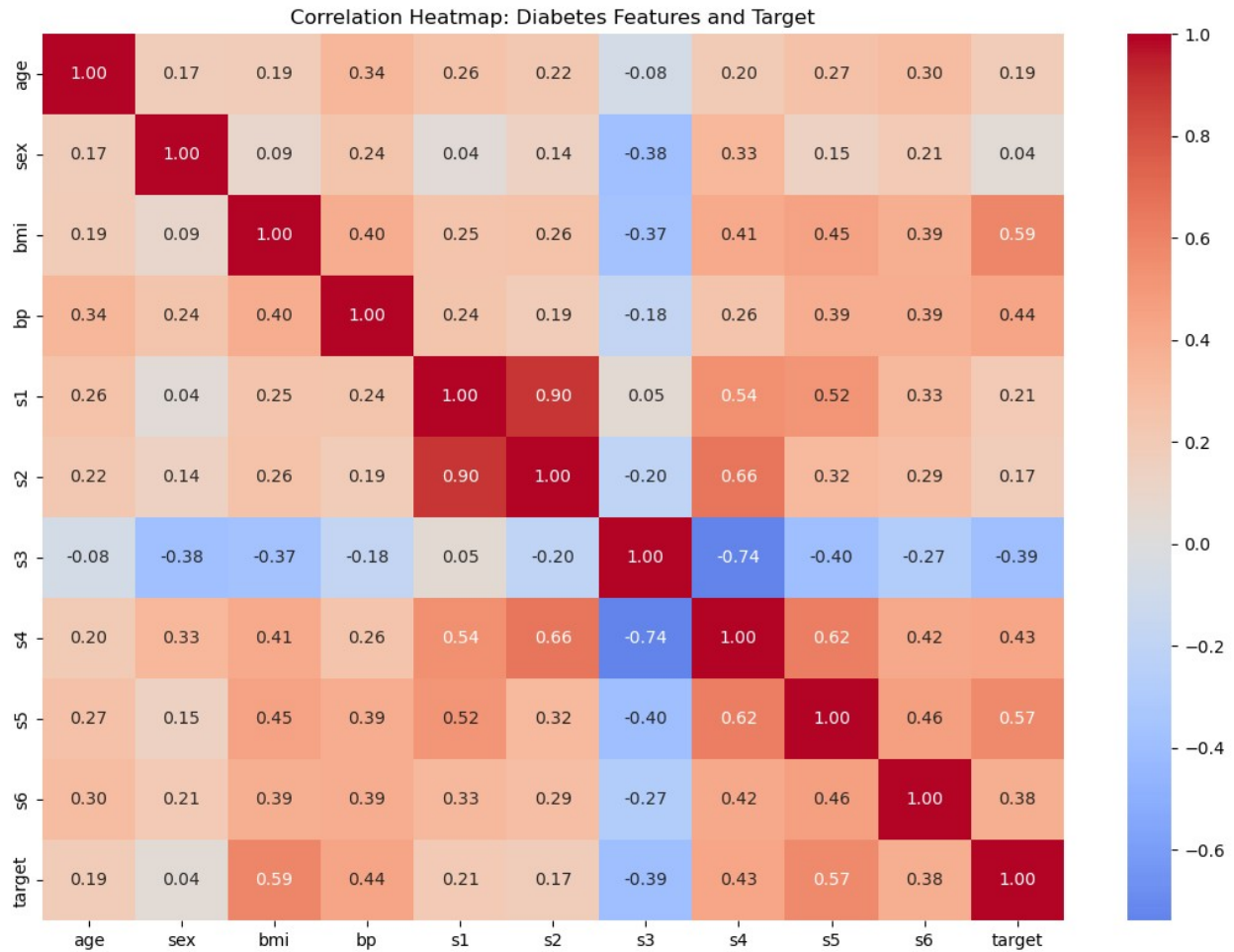
- Scatter plot (predicted vs. actual).
 - Residual plot.
- Evaluate model performance with:
 - MSE
 - RMSE
 - R^2 score

Which independent variable contributes the most to predicting disease progression?

```
import seaborn as sns

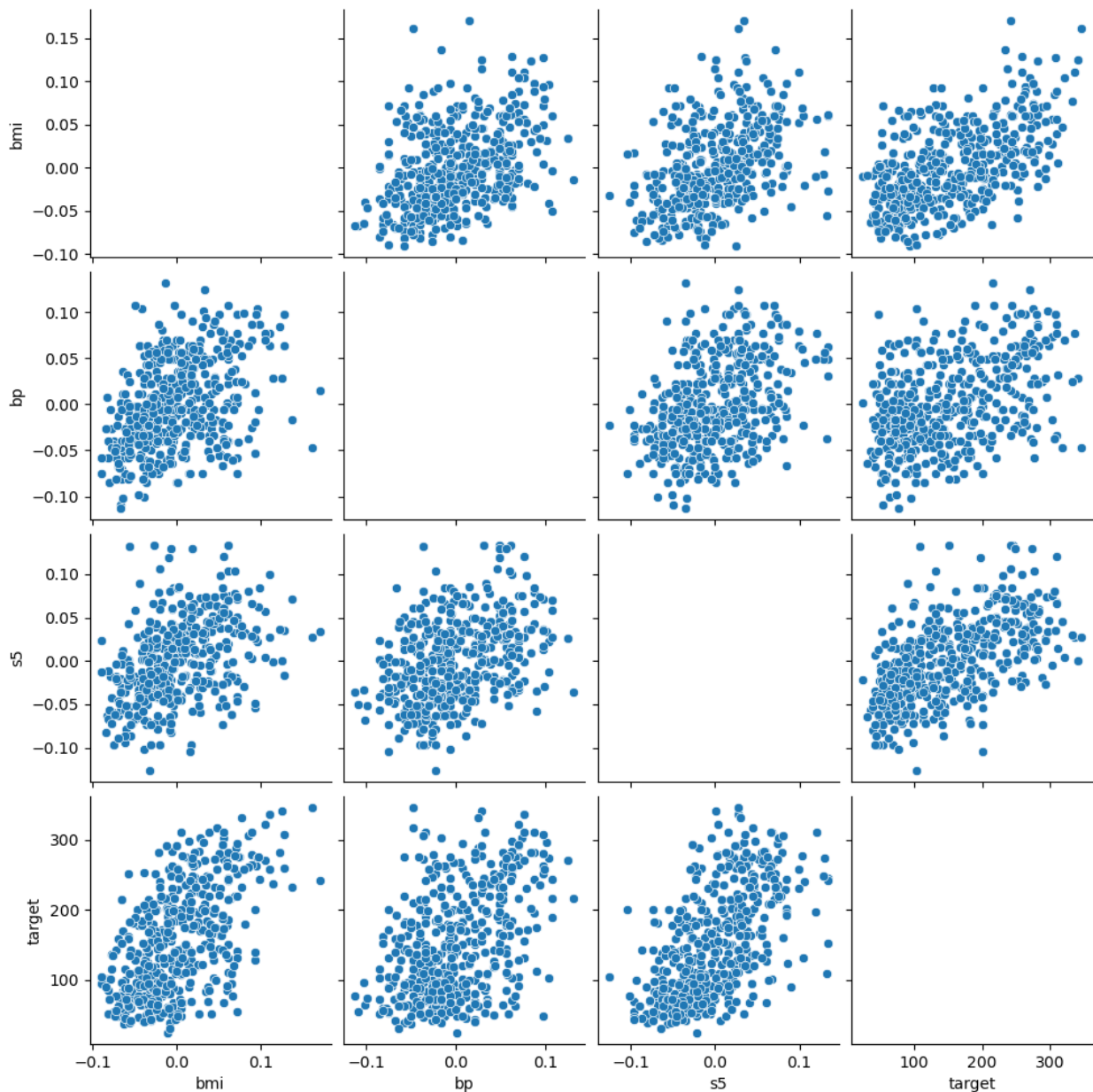
# Use all 10 features
feature_names = diabetes.feature_names
X = df[feature_names]
y_mlr = df["target"]

# Correlation heatmap
plt.figure(figsize=(11, 8))
sns.heatmap(
    df[feature_names + ["target"]].corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    center=0
)
plt.title("Correlation Heatmap: Diabetes Features and Target")
plt.tight_layout()
plt.show()
```



```
# Pair plots for selected features
selected_features = ["bmi", "bp", "s5", "target"]
sns.pairplot(df[selected_features], diag_kind="hist")
plt.suptitle("Selected Features vs. Disease Progression", y=1.02)
plt.show()
```

Selected Features vs. Disease Progression



```
# Split the data
X_train_mlr, X_test_mlr, y_train_mlr, y_test_mlr = train_test_split(
    X, y_mlr, test_size=0.2, random_state=42
)

# Train the multivariate linear regression model
mlr_model = LinearRegression()
mlr_model.fit(X_train_mlr, y_train_mlr)

LinearRegression()
```

```

# Predictions
y_pred_mlr = mlr_model.predict(X_test_mlr)
residuals_mlr = y_test_mlr - y_pred_mlr

# Evaluation metrics
mse_mlr = mean_squared_error(y_test_mlr, y_pred_mlr)
rmse_mlr = np.sqrt(mse_mlr)
r2_mlr = r2_score(y_test_mlr, y_pred_mlr)
print(f"Mean Squared Error (MSE): {mse_mlr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_mlr:.2f}")
print(f"R-squared Score (R²): {r2_mlr:.4f}")

Mean Squared Error (MSE): 2900.19
Root Mean Squared Error (RMSE): 53.85
R-squared Score (R²): 0.4526

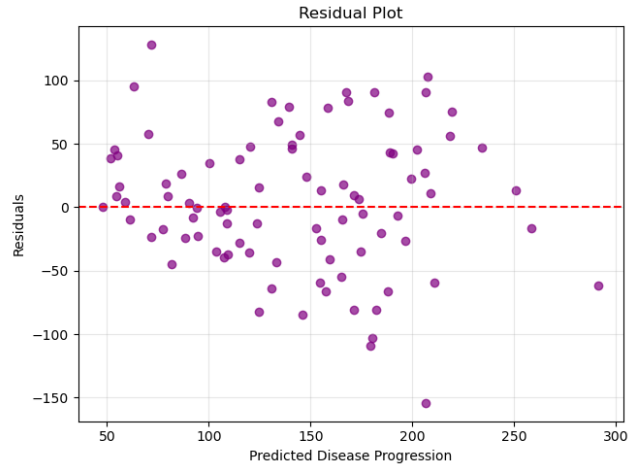
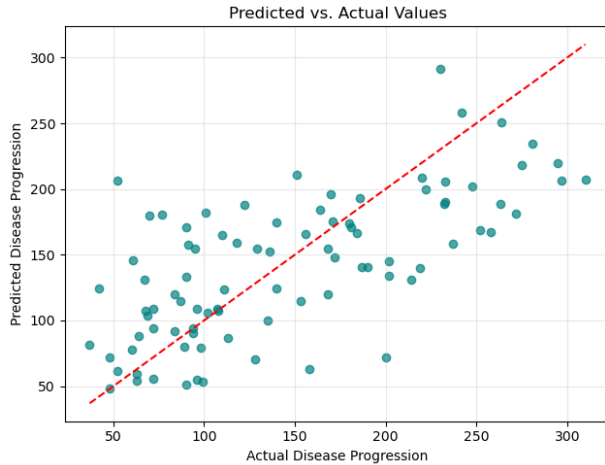
# Predicted vs. actual and residual plots
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].scatter(y_test_mlr, y_pred_mlr, alpha=0.7, color="teal")
axes[0].plot(
    [y_test_mlr.min(), y_test_mlr.max()],
    [y_test_mlr.min(), y_test_mlr.max()],
    "r--"
)
axes[0].set_title("Predicted vs. Actual Values")
axes[0].set_xlabel("Actual Disease Progression")
axes[0].set_ylabel("Predicted Disease Progression")
axes[0].grid(alpha=0.3)

axes[1].scatter(y_pred_mlr, residuals_mlr, alpha=0.7, color="purple")
axes[1].axhline(0, color="red", linestyle="--")
axes[1].set_title("Residual Plot")
axes[1].set_xlabel("Predicted Disease Progression")
axes[1].set_ylabel("Residuals")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

```



```
# Feature coefficient analysis
coefficient_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": mlr_model.coef_,
    "Absolute Coefficient": np.abs(mlr_model.coef_)
}).sort_values("Absolute Coefficient", ascending=False)

print("\nFeature Contributions:")
display(coefficient_df)

most_influential_feature = coefficient_df.iloc[0]["Feature"]
print(
    f"The independent variable with the largest contribution based on the "
    f"absolute regression coefficient is: {most_influential_feature}"
)
```

Feature Contributions:

	Feature	Coefficient	Absolute Coefficient
4	s1	-931.488846	931.488846
8	s5	736.198859	736.198859
2	bmi	542.428759	542.428759
5	s2	518.062277	518.062277
3	bp	347.703844	347.703844
7	s4	275.317902	275.317902
1	sex	-241.964362	241.964362
6	s3	163.419983	163.419983
9	s6	48.670657	48.670657
0	age	37.904021	37.904021

The independent variable with the largest contribution based on the absolute regression coefficient is: s1

Task 3: Experimentation & Model Comparison

- Compare performance of simple vs. multivariate regression in terms of evaluation metrics.

```
# Compare model performance
comparison_df = pd.DataFrame({
    "Model": [
        "Simple Linear Regression (BMI only)",
        "Multivariate Linear Regression (All 10 Features)"
    ],
    "MSE": [mse_slr, mse_mlr],
    "RMSE": [rmse_slr, rmse_mlr],
    "R²": [r2_slr, r2_mlr],
    "Variance Explained (%)": [r2_slr * 100, r2_mlr * 100]
})

display(comparison_df.style.format({
    "MSE": "{:.2f}",
    "RMSE": "{:.2f}",
    "R²": "{:.4f}",
    "Variance Explained (%)": "{:.2f}%"
}))

print(
    f"Multivariate regression performs better: RMSE decreases from "
    f"{rmse_slr:.2f} to {rmse_mlr:.2f}, while R² increases from "
    f"{r2_slr:.4f} to {r2_mlr:.4f}."
)

<pandas.io.formats.style.Styler at 0x172b7bcd940>

Multivariate regression performs better: RMSE decreases from 63.73 to
53.85, while R² increases from 0.2334 to 0.4526.
```

LAB # 03

LOGISTIC REGRESSION

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0-1 marks)	Marks (out of 20)
Understanding of Concept (4 marks)	Demonstrates clear and in-depth understanding of theory; strongly relates it well to lab objectives.	Basic understanding of theory; provides minimal or partial connection to lab objectives.	Little or no understanding of concept; unclear or incorrect relevance to lab topic.	
Code Implementation (6 marks)	Code is well-structured, correct, error-free, and reflects the theoretical concepts; handles edge cases effectively.	Code works with minor errors or inefficiencies; shows partial understanding of the concept.	Code is incorrect, incomplete, or does not align with lab goals.	
Use of Programming Features/Tools (4 marks)	Efficiently applies relevant Python libraries, data processing methods, and ML techniques (e.g., NumPy, Pandas, scikit-learn, Matplotlib) to achieve the desired outcomes.	Uses standard/basic functions and logic; limited or partial use of available features/tools.	Minimal or incorrect use of tools; relies on trivial constructs or hard-coded approaches.	
Results and Report (6 marks)	Produces accurate and well-presented results (visualizations, metrics, comparisons) with clear interpretation and insights.	Results are partially correct or lack clarity in interpretation; report is adequate but lacks depth, formatting, or coherence.	Results are missing, incorrect, or poorly explained.	

LAB # 03

LOGISTIC REGRESSION

OBJECTIVE

- To understand the concept of Logistic Regression and implement Logistic Regression for binary classification.

THEORY:

Practical Significance

Logistic Regression is one of the most widely used **classification algorithms** in machine learning. It is useful in applications such as:

- **Medical Diagnosis** (predicting if a patient has a disease based on symptoms).
- **Spam Detection** (classifying emails as spam or not spam).
- **Credit Scoring** (predicting if a customer will default on a loan).
- **Customer Churn Prediction** (identifying customers who are likely to leave a service).

Minimum Theoretical Background

1. Why Logistic Regression?

- Linear Regression is not ideal for classification because it produces continuous values.
- Logistic Regression **maps predictions to probabilities** between **0 and 1** using the **sigmoid function**.
- It is commonly used for **binary classification** problems.

2. Sigmoid Function

The **sigmoid function** converts a real number into a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = wX + b$.

3. Decision Rule

- If $\sigma(z) \geq 0.5 \rightarrow$ Predict Class 1.
- If $\sigma(z) < 0.5 \rightarrow$ Predict Class 0.

LAB TASKS

· Import Required Libraries

· Load Dataset

- Use a dataset **Breast Cancer Dataset** from `sklearn.datasets`.

· Exploratory Data Analysis (EDA)

- Display first few rows of the dataset.

- Check for missing values.
- Plot histograms or distribution of features.
- Check class distribution (malignant vs. benign).

· **Data Preprocessing**

- Split features and labels.
- Standardize features using StandardScaler.
- Train-test split.

· **Model Implementation**

- Use `sklearn.linear_model.LogisticRegression`.
- Train the model on the training data.
- Predict on the test data.

· **Evaluation**

- Accuracy score.
- Confusion matrix.
- Precision, Recall, F1-score.

LAB OUTCOMES

By completing this lab, students will:

- Understand the working of Logistic Regression.
- Learn how to apply Logistic Regression for binary classification.
- Evaluate classification models using accuracy, precision, recall, and F1-score.
- Gain hands-on experience in evaluating classification models.

Lab # 03: Logistic Regression

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Tasks:

- Import Required Libraries
- Load Dataset
 - Use a dataset Breast Cancer Dataset from sklearn.datasets.
- Exploratory Data Analysis (EDA)
 - Display first few rows of the dataset.
 - Check for missing values.
 - Plot histograms or distribution of features.
 - Check class distribution (malignant vs. benign).
- Data Preprocessing
 - Split features and labels.
 - Standardize features using StandardScaler.
 - Train-test split.
- Model Implementation
 - Use sklearn.linear_model.LogisticRegression.
 - Train the model on the training data.
 - Predict on the test data.
- Evaluation
 - Accuracy score.
 - Confusion matrix.
 - Precision, Recall, F1-score

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load dataset
cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = pd.Series(cancer.target, name="target")
```

```
# Display first few rows and check missing values
display(X.head())
print("Missing values:", X.isnull().sum().sum())
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst radius	worst texture	worst perimeter \
0	0.07871	...	25.38	17.33	184.60
1	0.05667	...	24.99	23.41	158.80
2	0.05999	...	23.57	25.53	152.50
3	0.09744	...	14.91	26.50	98.87
4	0.05883	...	22.54	16.67	152.20

	worst area	worst smoothness	worst compactness	worst concavity \
0	2019.0	0.1622	0.6656	0.7119
1	1956.0	0.1238	0.1866	0.2416
2	1709.0	0.1444	0.4245	0.4504
3	567.7	0.2098	0.8663	0.6869
4	1575.0	0.1374	0.2050	0.4000

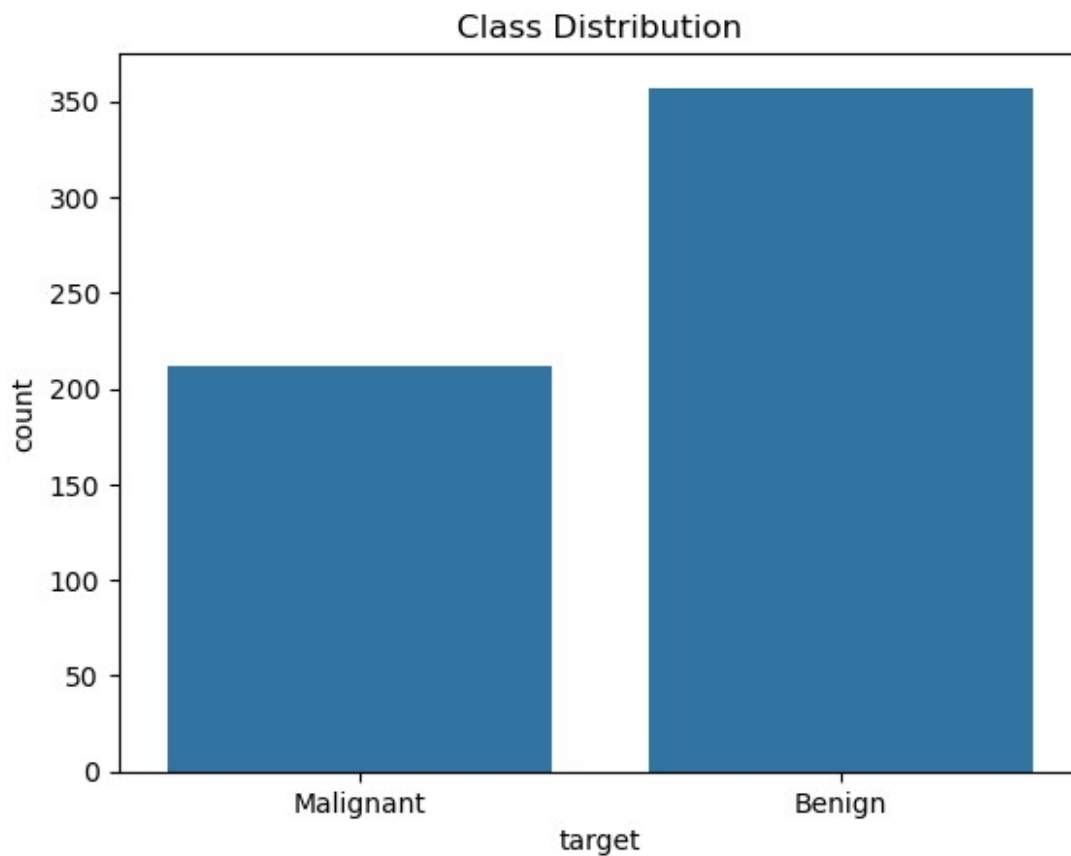
	worst concave points	worst symmetry	worst fractal dimension
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

Missing values: 0

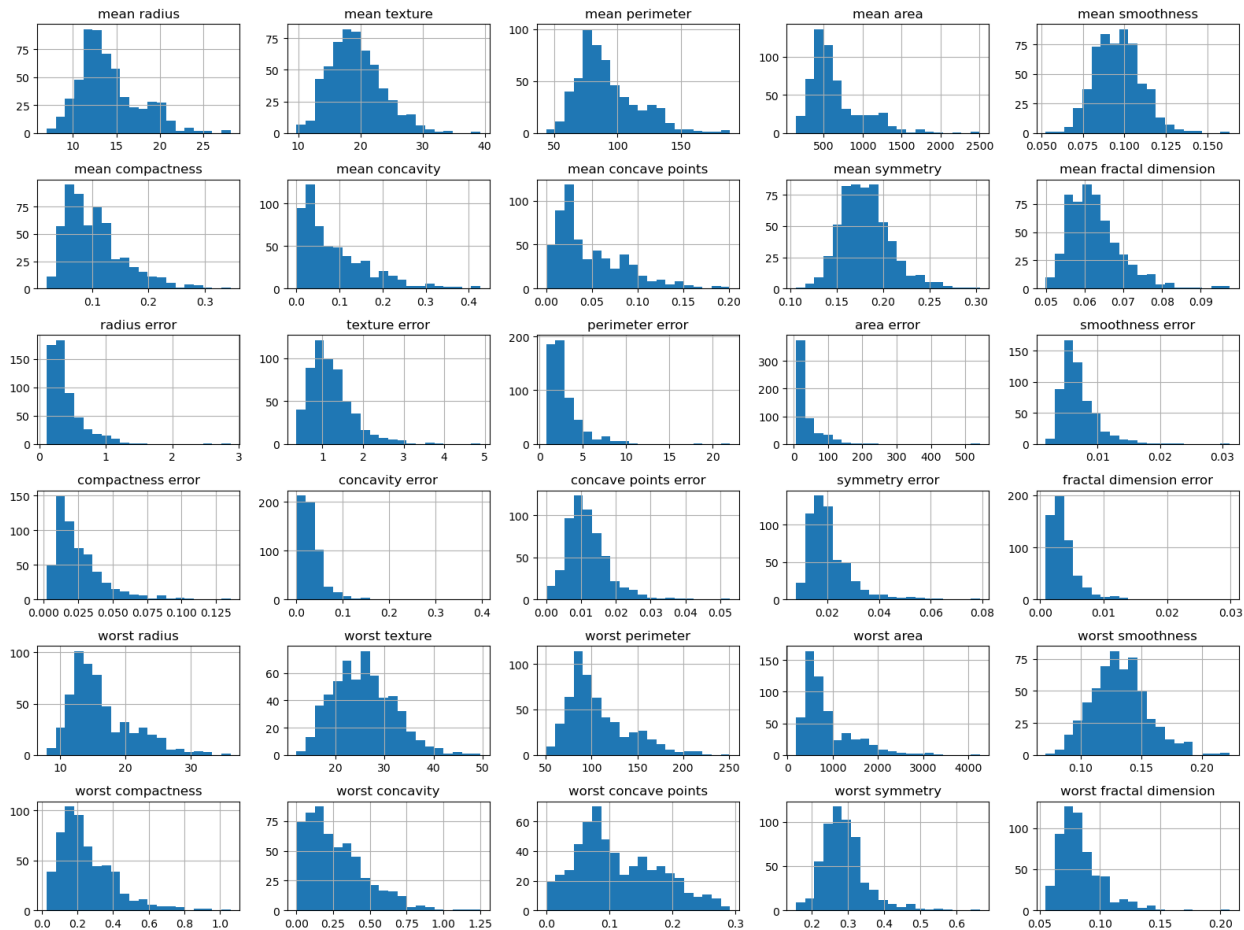
Class distribution

```
sns.countplot(x=y.map({0: "Malignant", 1: "Benign"}))
plt.title("Class Distribution")
plt.show()
```



Feature distributions

```
X.hist(figsize=(16, 12), bins=20)
plt.tight_layout()
plt.show()
```



```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train logistic regression model
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)

LogisticRegression(max_iter=1000, random_state=42)

# Predictions
y_pred = model.predict(X_test_scaled)

# Evaluation
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
```

```
print(classification_report(
    y_test,
    y_pred,
    target_names=["Malignant", "Benign"]
))
```

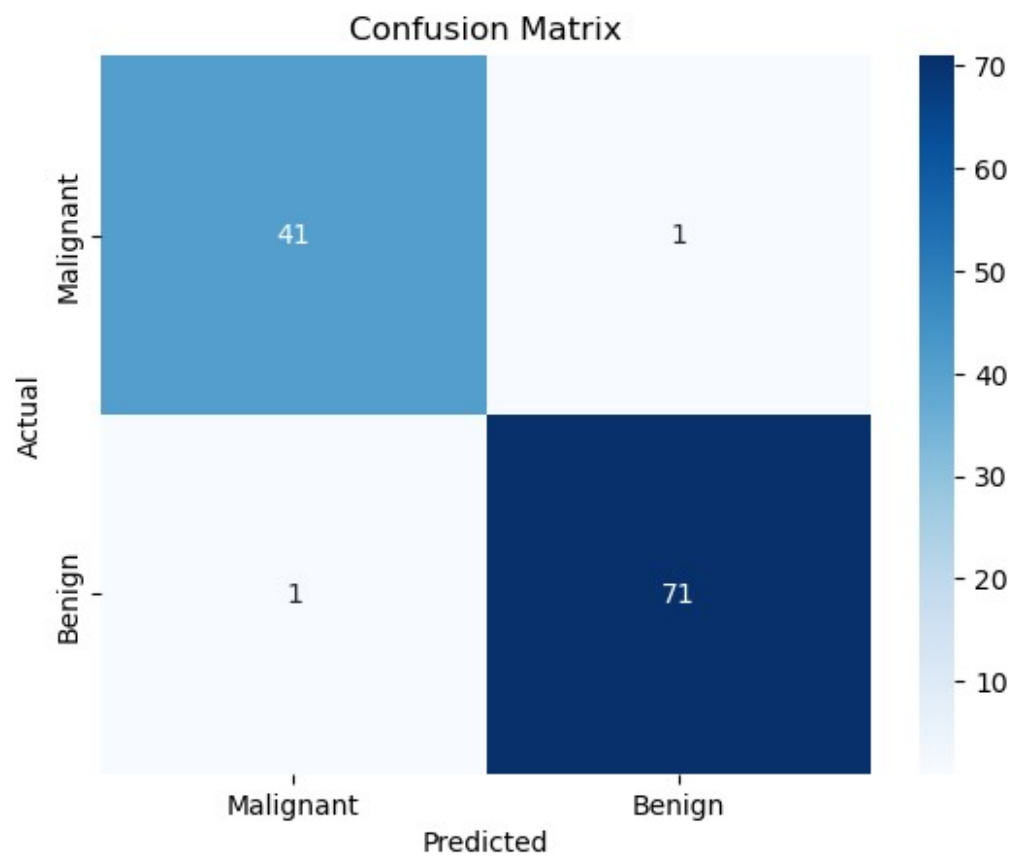
Accuracy: 0.9825

Classification Report:

	precision	recall	f1-score	support
Malignant	0.98	0.98	0.98	42
Benign	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

Confusion matrix

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Malignant", "Benign"],
    yticklabels=["Malignant", "Benign"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



LAB # 04

DECISION TREE CLASSIFIER

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0-1 marks)	Marks (out of 20)
Understanding of Concept (4 marks)	Demonstrates clear and in-depth understanding of theory; strongly relates it well to lab objectives.	Basic understanding of theory; provides minimal or partial connection to lab objectives.	Little or no understanding of concept; unclear or incorrect relevance to lab topic.	
Code Implementation (6 marks)	Code is well-structured, correct, error-free, and reflects the theoretical concepts; handles edge cases effectively.	Code works with minor errors or inefficiencies; shows partial understanding of the concept.	Code is incorrect, incomplete, or does not align with lab goals.	
Use of Programming Features/Tools (4 marks)	Efficiently applies relevant Python libraries, data processing methods, and ML techniques (e.g., NumPy, Pandas, scikit-learn, Matplotlib) to achieve the desired outcomes.	Uses standard/basic functions and logic; limited or partial use of available features/tools.	Minimal or incorrect use of tools; relies on trivial constructs or hard-coded approaches.	
Results and Report (6 marks)	Produces accurate and well-presented results (visualizations, metrics, comparisons) with clear interpretation and insights.	Results are partially correct or lack clarity in interpretation; report is adequate but lacks depth, formatting, or coherence.	Results are missing, incorrect, or poorly explained.	

LAB # 04

DECISION TREE CLASSIFIER

OBJECTIVE

To understand and implement a Decision Tree Classifier for classification tasks.

THEORY:

Practical Significance

Decision Tree classifiers are widely used for classification problems because they:

- Are **easy to interpret** and visualize.
- Work well with **both numerical and categorical data**.
- Do not require feature scaling.
- Can **handle missing values** better than other models.
- Are used in real-world applications like:
 - **Medical diagnosis** (e.g., predicting disease based on symptoms).
 - **Fraud detection** (e.g., detecting fraudulent transactions).
 - **Customer segmentation** (e.g., classifying customers based on purchasing behavior).

Minimum Theoretical Background

1. How Decision Trees Work

- Decision trees split the dataset **recursively** into smaller subsets.
- The splits are based on **feature values** that minimize impurity.
- Each internal node represents a **decision rule**, and each leaf node represents a **class label**.

2. Splitting Criteria

The algorithm selects the best feature to split the data based on impurity measures:

- **Gini Index**: Measures impurity using the formula:

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

where p_i is the probability of class i .

- **Entropy (Information Gain)**: Measures impurity using:

$$Entropy = - \sum_{i=1}^c p_i \log_2 p_i$$

where p_i is the probability of class i .

- The feature that provides the **highest Information Gain** or **lowest Gini Index** is chosen for splitting.

3. Overfitting and Pruning

- **Overfitting** occurs when the tree is too deep and fits noise in the training data.
- **Pruning** is used to **reduce tree complexity** and improve generalization.
- Techniques include **pre-pruning** (limiting tree depth) and **post-pruning** (removing unnecessary branches).

LAB TASKS

Task 1: Load Dataset

- Use a dataset such as **Iris dataset** (from sklearn) or load any CSV file.

Task 2: Exploratory Data Analysis (EDA)

- Display dataset information, shape, and statistical summary.
- Plot distributions of features (histograms / pairplot).
- Visualize correlations

Task 3: Data Preprocessing

- Split dataset into training and testing sets (e.g., 70% train, 30% test).

Task 4: Build Decision Tree Classifier

- Train the model using DecisionTreeClassifier.

Task 5: Model Evaluation

- Make predictions on the test data.
- Evaluate using:
 - Accuracy Score
 - Confusion Matrix
 - Classification Report

Task 6: Experimentation

- Change parameters (criterion = "gini" vs "entropy", max_depth, min_samples_split).
- Compare results and observe **overfitting vs underfitting**.

LAB OUTCOMES

By completing this lab, students will:

- Understand the working of Decision Tree Classifier.
- Apply Gini Index and Entropy for splitting criteria.
- Gain hands-on experience in evaluating classification models.

Lab # 04: Decision Tree Classifier

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Task 1: Load Dataset

- Use a dataset such as Iris dataset (from sklearn) or load any CSV file.

```
# importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

# loading iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
display(df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	
0.2				
1	4.9	3.0	1.4	
0.2				
2	4.7	3.2	1.3	
0.2				
3	4.6	3.1	1.5	
0.2				
4	5.0	3.6	1.4	
0.2				

	target	species
0	0	setosa
1	0	setosa
2	0	setosa

```
3      0  setosa
4      0  setosa
```

Task 2: Exploratory Data Analysis (EDA)

- Display dataset information, shape, and statistical summary.
- Plot distributions of features (histograms / pairplot).
- Visualize correlations

```
# shape of dataset
print("Dataset shape:", df.shape)
```

Dataset shape: (150, 6)

```
# dataset information
print("Dataset info:", df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   target                 150 non-null   int64
5   species                150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
Dataset info: None
```

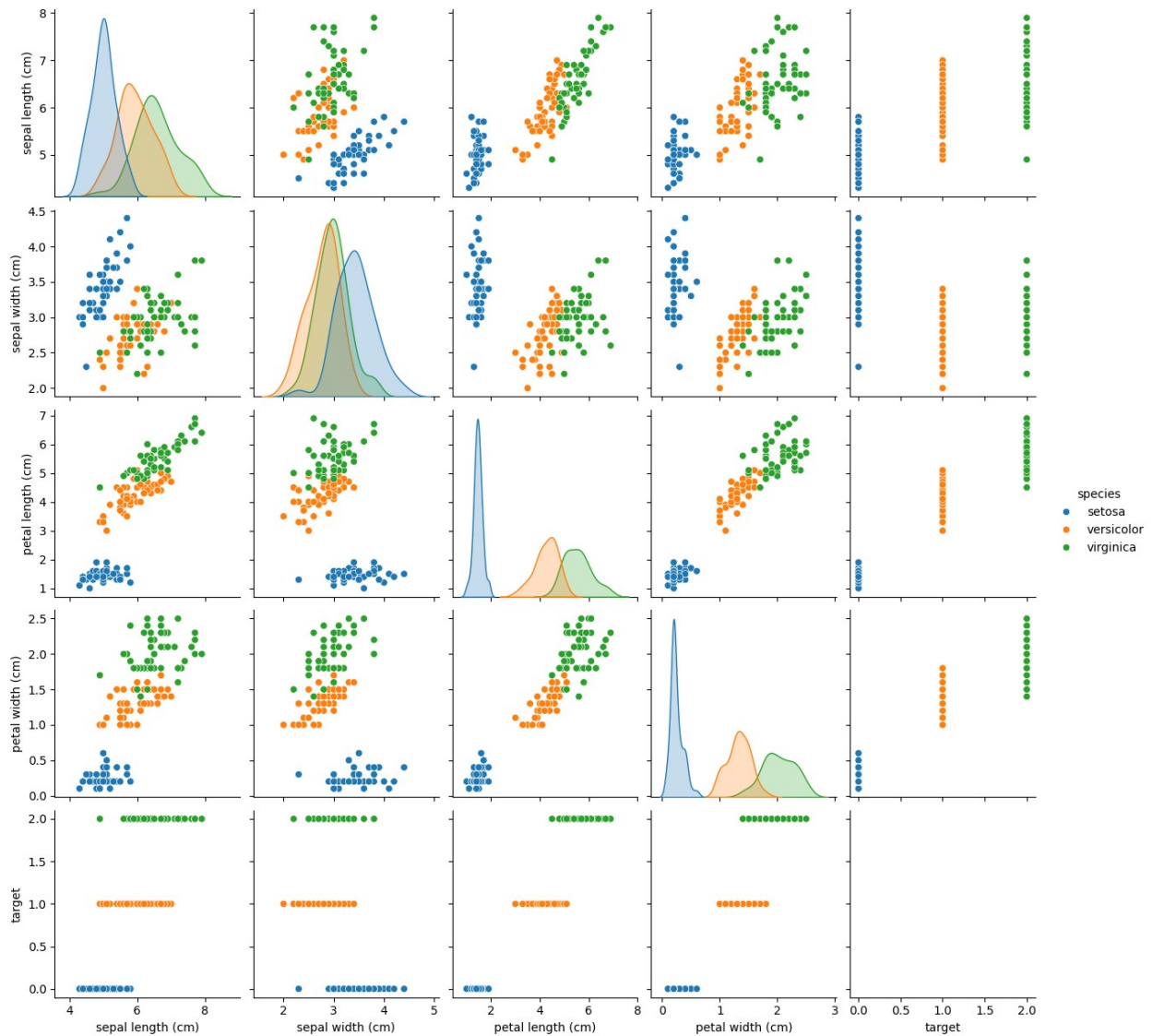
```
# statistical summary of dataset
print("Dataset description:", df.describe())
```

```
Dataset description:      sepal length (cm)  sepal width (cm)  petal
length (cm) \
count      150.000000      150.000000      150.000000
mean         5.843333         3.057333         3.758000
std          0.828066         0.435866         1.765298
min          4.300000         2.000000         1.000000
25%          5.100000         2.800000         1.600000
50%          5.800000         3.000000         4.350000
75%          6.400000         3.300000         5.100000
max          7.900000         4.400000         6.900000

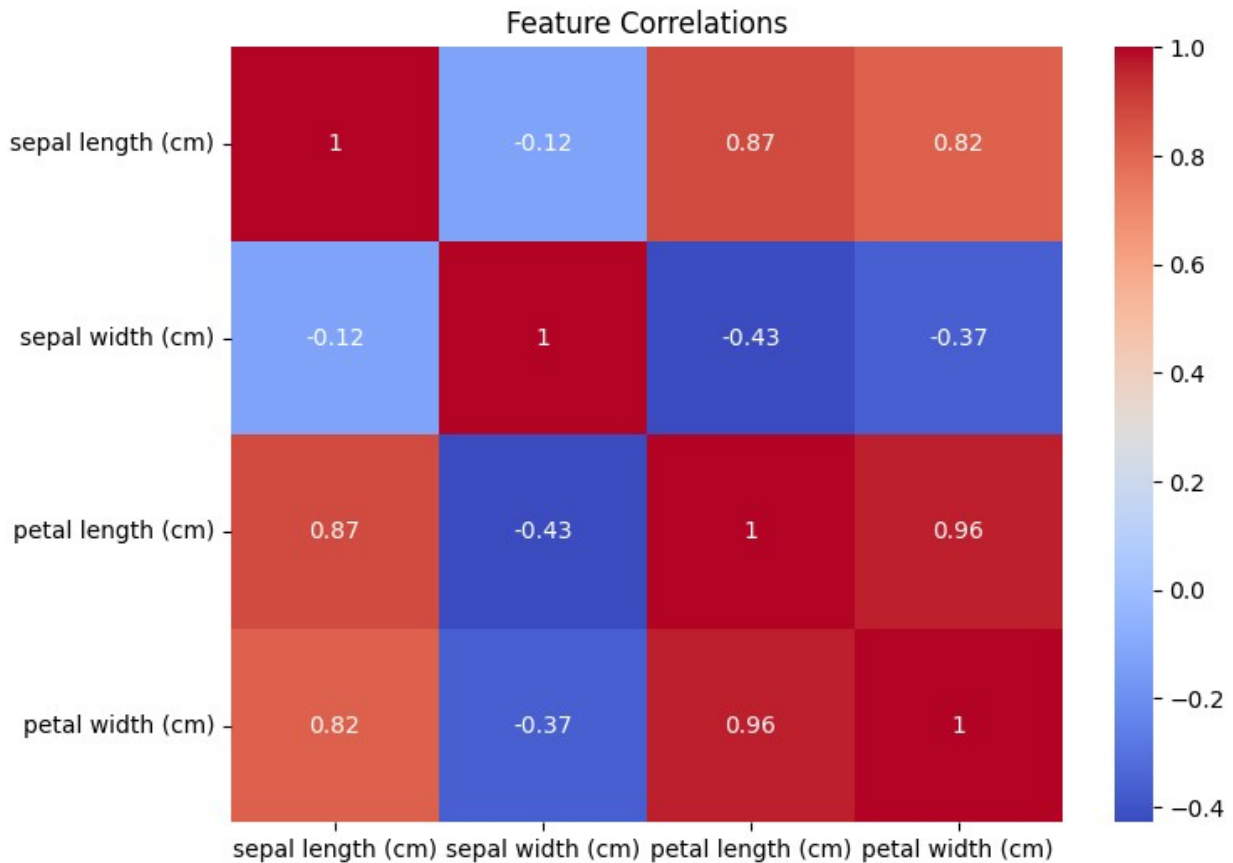
      petal width (cm)  target
count      150.000000  150.000000
mean         1.199333    1.000000
std          0.762238    0.819232
min          0.100000    0.000000
```

25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

```
# plotting a histogram of the features
sns.pairplot(df, hue="species")
plt.show()
```



```
# heatmap of feature correlations
plt.figure(figsize=(8, 6))
sns.heatmap(df[iris.feature_names].corr(), annot=True,
            cmap="coolwarm")
plt.title("Feature Correlations")
plt.show()
```



Task 3: Data Preprocessing

- Split dataset into training and testing sets (e.g., 70% train, 30% test).

```
# splitting the dataset into training and testing sets
X = df[iris.feature_names]
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

Task 4: Build Decision Tree Classifier

- Train the model using DecisionTreeClassifier.

```
# training the decision tree classifier
model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)

DecisionTreeClassifier(random_state=42)
```

Task 5: Model Evaluation

- Make predictions on the test data.
- Evaluate using:
 - Accuracy Score

- Confusion Matrix
- Classification Report

```
# making predictions on the test set
```

```
y_pred = model.predict(X_test)
```

```
# evaluating the model with accuracy and classification report
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
classification_rep = classification_report(y_test, y_pred,  
target_names=iris.target_names)
```

```
print("Classification Report:\n", classification_rep)
```

```
Accuracy: 1.0
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

```
# confusion matrix
```

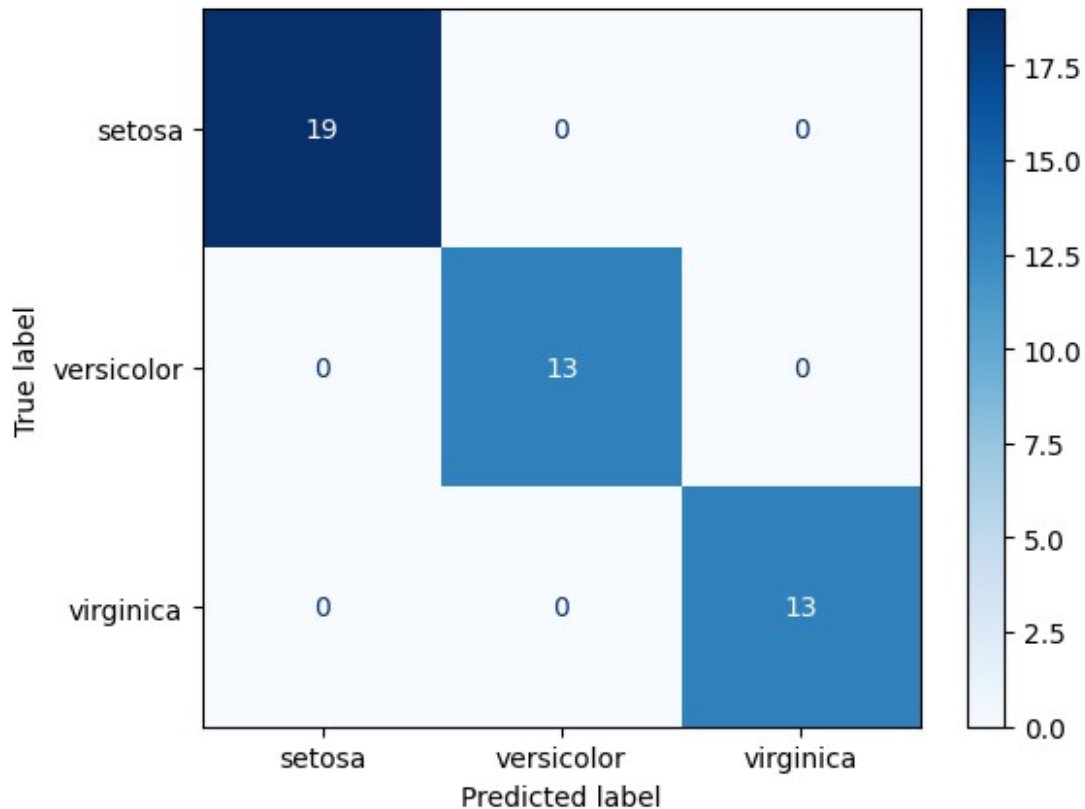
```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
display_labels=iris.target_names)
```

```
disp.plot(cmap=plt.cm.Blues)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at  
0x177a3e4aad0>
```



Task 6: Experimentation

- Change parameters (criterion = "gini" vs "entropy", max_depth, min_samples_split).
- Compare results and observe overfitting vs underfitting.

```
# training and testing accuracy with entropy
treeEntropy = DecisionTreeClassifier(criterion="entropy", max_depth=3,
min_samples_split=2, random_state=42)
treeEntropy.fit(X_train, y_train)
train_accuracy_entropy = treeEntropy.score(X_train, y_train)
test_accuracy_entropy = treeEntropy.score(X_test, y_test)
print("Training Accuracy with Entropy:", train_accuracy_entropy)
print("Testing Accuracy with Entropy:", test_accuracy_entropy)
```

```
Training Accuracy with Entropy: 0.9523809523809523
Testing Accuracy with Entropy: 0.9777777777777777
```

```
# training and testing accuracy with gini index
treeGini = DecisionTreeClassifier(criterion="gini", max_depth=3,
min_samples_split=2, random_state=42)
treeGini.fit(X_train, y_train)
train_accuracy_gini = treeGini.score(X_train, y_train)
test_accuracy_gini = treeGini.score(X_test, y_test)
print("Training Accuracy with Gini:", train_accuracy_gini)
print("Testing Accuracy with Gini:", test_accuracy_gini)
```

Training Accuracy with Gini: 0.9523809523809523
Testing Accuracy with Gini: 1.0

comparing the results of both models

```
results = {  
    "Model": ["Decision Tree (Entropy)", "Decision Tree (Gini)"],  
    "Training Accuracy": [train_accuracy_entropy,  
train_accuracy_gini],  
    "Testing Accuracy": [test_accuracy_entropy, test_accuracy_gini]  
}  
print("Results Comparison:\n", pd.DataFrame(results))
```

Results Comparison:

	Model	Training Accuracy	Testing Accuracy
0	Decision Tree (Entropy)	0.952381	0.977778
1	Decision Tree (Gini)	0.952381	1.000000

LAB # 05

RANDOM FOREST CLASSIFIER

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0-1 marks)	Marks (out of 20)
Understanding of Concept (4 marks)	Demonstrates clear and in-depth understanding of theory; strongly relates it well to lab objectives.	Basic understanding of theory; provides minimal or partial connection to lab objectives.	Little or no understanding of concept; unclear or incorrect relevance to lab topic.	
Code Implementation (6 marks)	Code is well-structured, correct, error-free, and reflects the theoretical concepts; handles edge cases effectively.	Code works with minor errors or inefficiencies; shows partial understanding of the concept.	Code is incorrect, incomplete, or does not align with lab goals.	
Use of Programming Features/Tools (4 marks)	Efficiently applies relevant Python libraries, data processing methods, and ML techniques (e.g., NumPy, Pandas, scikit-learn, Matplotlib) to achieve the desired outcomes.	Uses standard/basic functions and logic; limited or partial use of available features/tools.	Minimal or incorrect use of tools; relies on trivial constructs or hard-coded approaches.	
Results and Report (6 marks)	Produces accurate and well-presented results (visualizations, metrics, comparisons) with clear interpretation and insights.	Results are partially correct or lack clarity in interpretation; report is adequate but lacks depth, formatting, or coherence.	Results are missing, incorrect, or poorly explained.	

LAB # 05

RANDOM FOREST CLASSIFIER

OBJECTIVE

- To understand and Implement a **Random Forest Classifier** for classification tasks.

THEORY:

Practical Significance

Random Forest is an ensemble learning method that combines multiple decision trees to improve classification performance. It is widely used because:

- It **reduces overfitting** compared to individual decision trees.
- It is **robust to noise and missing data**.
- It is used in real-world applications like:
 - **Medical diagnosis** (e.g., detecting diseases).
 - **Fraud detection** (e.g., credit card fraud)..

Minimum Theoretical Background

1. How Random Forest Works

- Random Forest builds multiple decision trees from **random subsets** of data.
- Each tree makes a prediction, and the final prediction is based on a **majority vote** (classification) or an **average** (regression).

2. Key Features of Random Forest

- **Bootstrap Aggregating (Bagging)**: Each tree is trained on a different random subset of the data.
- **Feature Randomness**: At each split, only a **random subset** of features is considered.
- **Majority Voting**: The final prediction is based on the most common class predicted by individual trees.

3. Comparison with Decision Trees

- **Decision Trees** can overfit to training data, while **Random Forests** reduce overfitting.
- **Random Forests** provide better generalization and robustness.

Mathematical Expression

- Entropy (for information gain):

$$Entropy = - \sum_{i=1}^c p_i \log_2 p_i$$

- Gini Index:

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

LAB TASKS

1. Load a dataset for classification (e.g., Titanic, Breast Cancer dataset).
2. Apply data preprocessing (handle missing values, encode categorical data).
3. Split the dataset into training and testing sets.
4. Train a Random Forest Classifier on the training data.
5. Make predictions on the test set.
6. Evaluate performance using accuracy, precision, recall, and F1-score.
7. Visualize the Confusion Matrix
8. Compare with a Single Decision Tree

LAB OUTCOMES

By completing this lab, students will:

- Understand the working of Random Forest Classifier.
- Learn how bootstrap aggregating (bagging) works in Random Forest.
- Gain hands-on experience in evaluating classification models.

Lab # 05: Random Forest Classifier

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Lab Tasks:

1. Load a dataset for classification (e.g., Titanic, Breast Cancer dataset).

```
# importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, classification_report

# loading dataset and preparing features and target
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')
print(f"Dataset Shape: {X.shape}")
print(f"Target distribution (0: Malignant, 1: Benign):\n{y.value_counts()}\n")

Dataset Shape: (569, 30)
Target distribution (0: Malignant, 1: Benign):
target
1    357
0    212
Name: count, dtype: int64
```

1. Apply data preprocessing (handle missing values, encode categorical data).

```
# handling missing values
missing_vals = X.isnull().sum().sum()
print(f"Total missing values found: {missing_vals}")

Total missing values found: 0
```

1. Split the dataset into training and testing sets.

```
# splitting data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}\n")
```

```
Training set shape: (398, 30)
Testing set shape: (171, 30)
```

1. Train a Random Forest Classifier on the training data.

```
# training a random forest classifier
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train, y_train)

RandomForestClassifier(random_state=42)
```

1. Make predictions on the test set.

```
# making predictions
rf_preds = rf_clf.predict(X_test)
```

1. Evaluate performance using accuracy, precision, recall, and F1-score.

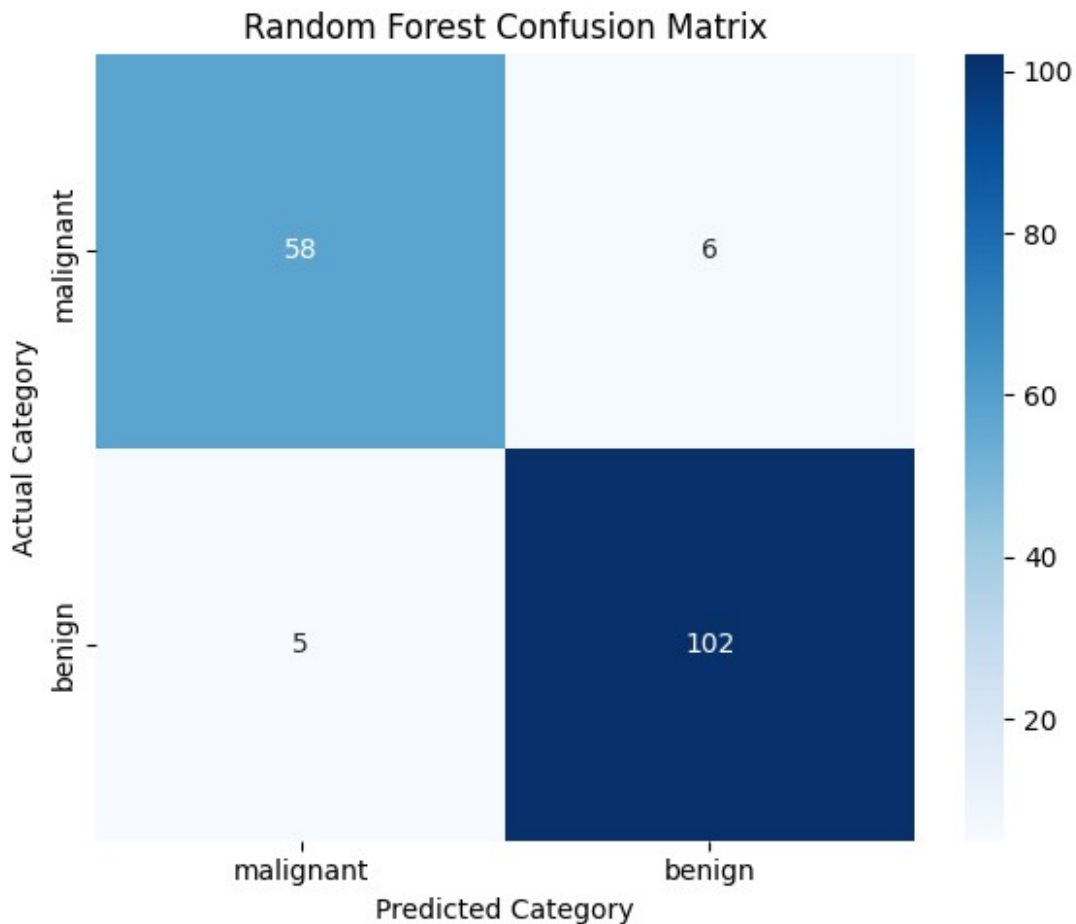
```
rf_accuracy = accuracy_score(y_test, rf_preds)
rf_precision = precision_score(y_test, rf_preds)
rf_recall = recall_score(y_test, rf_preds)
rf_f1 = f1_score(y_test, rf_preds)
print(f"Accuracy: {rf_accuracy:.4f}")
print(f"Precision: {rf_precision:.4f}")
print(f"Recall: {rf_recall:.4f}")
print(f"F1-Score: {rf_f1:.4f}\n")
```

```
Accuracy: 0.9357
Precision: 0.9444
Recall: 0.9533
F1-Score: 0.9488
```

1. Visualize the Confusion Matrix

```
# confusion matrix
rf_cm = confusion_matrix(y_test, rf_preds)
plt.figure(figsize=(6, 5))
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names,
            yticklabels=data.target_names)
plt.title('Random Forest Confusion Matrix')
plt.ylabel('Actual Category')
```

```
plt.xlabel('Predicted Category')
plt.tight_layout()
plt.show()
```



1. Compare with a Single Decision Tree

```
# comparing with a single decision tree classifier
dt_clf = DecisionTreeClassifier(random_state=42)
dt_clf.fit(X_train, y_train)
dt_preds = dt_clf.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_preds)
dt_precision = precision_score(y_test, dt_preds)
dt_recall = recall_score(y_test, dt_preds)
dt_f1 = f1_score(y_test, dt_preds)
comparison_df = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score'],
    'Single Decision Tree': [dt_accuracy, dt_precision, dt_recall,
dt_f1],
    'Random Forest': [rf_accuracy, rf_precision, rf_recall, rf_f1]
})
print(comparison_df.to_string(index=False))
```

Metric	Single Decision Tree	Random Forest
Accuracy	0.918129	0.935673
Precision	0.934579	0.944444
Recall	0.934579	0.953271
F1-Score	0.934579	0.948837

LAB # 06

SUPPORT VECTOR MACHINE (SVM) CLASSIFIER

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0-1 marks)	Marks (out of 20)
Understanding of Concept (4 marks)	Demonstrates clear and in-depth understanding of theory; strongly relates it well to lab objectives.	Basic understanding of theory; provides minimal or partial connection to lab objectives.	Little or no understanding of concept; unclear or incorrect relevance to lab topic.	
Code Implementation (6 marks)	Code is well-structured, correct, error-free, and reflects the theoretical concepts; handles edge cases effectively.	Code works with minor errors or inefficiencies; shows partial understanding of the concept.	Code is incorrect, incomplete, or does not align with lab goals.	
Use of Programming Features/Tools (4 marks)	Efficiently applies relevant Python libraries, data processing methods, and ML techniques (e.g., NumPy, Pandas, scikit-learn, Matplotlib) to achieve the desired outcomes.	Uses standard/basic functions and logic; limited or partial use of available features/tools.	Minimal or incorrect use of tools; relies on trivial constructs or hard-coded approaches.	
Results and Report (6 marks)	Produces accurate and well-presented results (visualizations, metrics, comparisons) with clear interpretation and insights.	Results are partially correct or lack clarity in interpretation; report is adequate but lacks depth, formatting, or coherence.	Results are missing, incorrect, or poorly explained.	

LAB # 06

SUPPORT VECTOR MACHINE (SVM) CLASSIFIER

OBJECTIVE

To understand the Support Vector Machine (SVM) Classifier algorithm and implement an SVM classifier for binary classification.

THEORY:

Practical Significance

Machine learning models require numerical input. Since categorical data consists of non-numeric

Practical Significance

Support Vector Machines (SVM) are widely used in classification tasks due to their **ability to handle high-dimensional data and small sample sizes**. Some real-world applications include:

- **Face recognition**
- **Text classification** (spam detection, sentiment analysis)
- **Bioinformatics** (gene classification)
- **Medical diagnosis**

Minimum Theoretical Background

1. How SVM Works

- SVM finds the **optimal hyperplane** that maximizes the margin between two classes.
- The points that **define the margin** are called **support vectors**.

2. Kernel Trick

- When data is not linearly separable, **kernel functions** transform it into a higher-dimensional space where it becomes separable.
- Common kernels:
 - **Linear Kernel:** $K(x_i, x_j) = x_i^T x_j$
 - **Polynomial Kernel:** $K(x_i, x_j) = (x_i^T x_j + c)^d$
 - **Radial Basis Function (RBF) Kernel:** $K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$

3. Soft Margin & C Parameter

- **Hard Margin:** Strict separation (only for perfectly separable data).
- **Soft Margin:** Allows some misclassifications, controlled by **C** (regularization parameter).

LAB TASKS

1. Load a dataset for classification (e.g., Parkinson disease, Breast Cancer dataset).
2. Apply data preprocessing (handle missing values, encode categorical data).
3. Split the dataset into training and testing sets.
4. Apply Grid search to find the optimal parameters
5. Use those parameters to make predictions on the test set.
6. Evaluate performance using accuracy, precision, recall, and F1-score.
7. Visualize the Confusion Matrix

LAB OUTCOMES

By completing this lab, students will:

- Understand Support Vector Machine (SVM) Classifier.
- Learn about different kernel functions in SVM.
- Gain hands-on experience in evaluating classification models.

Lab # 06: Support Vector Machine (SVM) Classifier

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Lab Tasks:

1. Load a dataset for classification (e.g., Parkinson disease, Breast Cancer dataset).

```
# importing libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")
print("Features:")
print(X.head())
print("\nTarget:")
print(y.head())
print("\nDataset Shape:", X.shape)
```

```
Features:
   mean radius  mean texture  mean perimeter  mean area  mean
smoothness \
0          17.99         10.38          122.80      1001.0
0.11840
1          20.57         17.77          132.90      1326.0
0.08474
2          19.69         21.25          130.00      1203.0
0.10960
3          11.42         20.38           77.58       386.1
0.14250
4          20.29         14.34          135.10      1297.0
```

0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst radius	worst texture	worst perimeter \
0	0.07871	...	25.38	17.33	184.60
1	0.05667	...	24.99	23.41	158.80
2	0.05999	...	23.57	25.53	152.50
3	0.09744	...	14.91	26.50	98.87
4	0.05883	...	22.54	16.67	152.20

	worst area	worst smoothness	worst compactness	worst concavity \
0	2019.0	0.1622	0.6656	0.7119
1	1956.0	0.1238	0.1866	0.2416
2	1709.0	0.1444	0.4245	0.4504
3	567.7	0.2098	0.8663	0.6869
4	1575.0	0.1374	0.2050	0.4000

	worst concave points	worst symmetry	worst fractal dimension
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

Target:

0 0
1 0
2 0
3 0
4 0

Name: target, dtype: int64

Dataset Shape: (569, 30)

1. Apply data preprocessing (handle missing values, encode categorical data).

```
# checking missing values
print("Total Missing Values:", X.isnull().sum().sum())
```

Total Missing Values: 0

1. Split the dataset into training and testing sets.

```
# train test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42, stratify=y)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

Training Data: (455, 30)

Testing Data: (114, 30)

1. Apply Grid search to find the optimal parameters

```
# creating a pipeline with StandardScaler and RandomForestClassifier
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC())
])
```

```
# parameters to test
param_grid = {
    "svm__C": [0.1, 1, 10, 100],
    "svm__kernel": ["linear", "rbf"],
    "svm__gamma": ["scale", "auto"]
}
```

```
# performing grid search with cross-validation
grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
```

```
# fitting the grid search to the training data
grid_search.fit(X_train, y_train)
```

```
GridSearchCV(cv=5,
              estimator=Pipeline(steps=[('scaler', StandardScaler())],
```

```

        ('svm', SVC()))],
        n_jobs=-1,
        param_grid={'svm__C': [0.1, 1, 10, 100],
                    'svm__gamma': ['scale', 'auto'],
                    'svm__kernel': ['linear', 'rbf']},
        scoring='accuracy')

# display best parameters
print("Best Parameters:")
print(grid_search.best_params_)
print("\nBest Cross-Validation Accuracy:")
print(grid_search.best_score_)

Best Parameters:
{'svm__C': 0.1, 'svm__gamma': 'scale', 'svm__kernel': 'linear'}

Best Cross-Validation Accuracy:
0.9780219780219781

```

1. Use those parameters to make predictions on the test set.

```

# evaluating the best model on the test set
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
print("Predictions:")
print(y_pred)

Predictions:
[0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0
 0 1
 1 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 1
 1 0
 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 1
 0 1
 0 1 1]

```

1. Evaluate performance using accuracy, precision, recall, and F1-score.

```

# calculating performance metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall    :", recall)
print("F1 Score  :", f1)

Accuracy : 0.9824561403508771
Precision: 0.9861111111111112

```

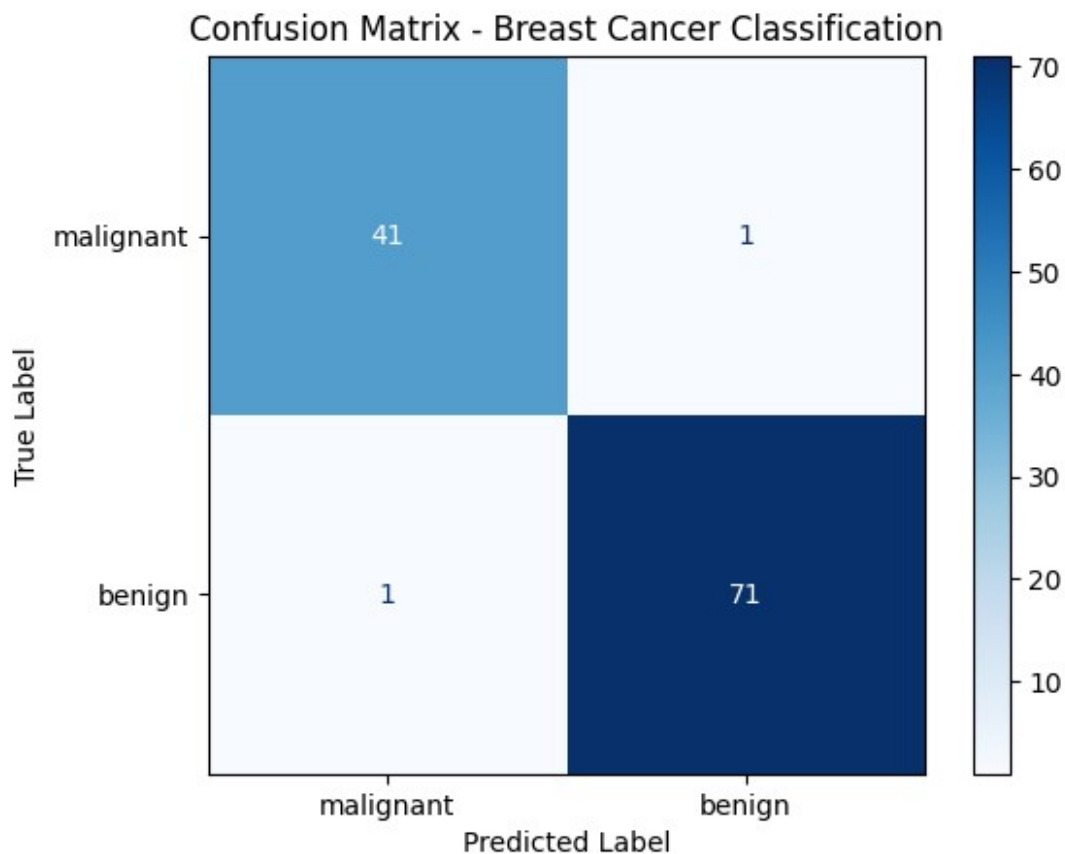
```
Recall    : 0.9861111111111112  
F1 Score  : 0.9861111111111112
```

1. Visualize the Confusion Matrix

```
# confusion matrix  
cm = confusion_matrix(y_test, y_pred)  
print("\nConfusion Matrix:")  
print(cm)  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=cm,  
    display_labels=data.target_names  
)  
disp.plot(cmap="Blues")  
plt.title("Confusion Matrix - Breast Cancer Classification")  
plt.xlabel("Predicted Label")  
plt.ylabel("True Label")  
plt.show()
```

Confusion Matrix:

```
[[41  1]  
 [ 1 71]]
```



LAB # 07
OPEN ENDED LAB

Performance Metric	Exceeds Expectations (5–4)	Meets Expectations (3–2)	Does Not Meet Expectations (0–1)	Marks (Out of 20)
Experiment Design (4 marks)	Designs an innovative and well-structured machine learning experiment addressing the open-ended challenge with multiple approaches and clear objectives.	Designs a functional machine learning experiment that addresses the challenge with moderate guidance or limited innovation.	Struggles to design an appropriate experiment or fails to address the challenge adequately.	
Implementation & Execution (6 marks)	Implements the experiment with high technical proficiency using appropriate ML tools and libraries, achieving accurate results supported by sound analysis.	Implements the experiment adequately with minor errors, incomplete testing, or limited analysis.	Implementation is incomplete, results are inaccurate, or lacks analysis.	
Problem Solving & Adaptability (4 marks)	Demonstrates strong analytical skills, creativity, and adaptability in handling challenges during experimentation and model optimization.	Solves problems adequately with occasional guidance or limited creativity.	Unable to effectively solve problems or adapt to issues during execution.	
Report & Presentation (6 marks)	Produces a comprehensive, well-organized report with clear explanations, visualizations, and properly formatted code; delivers an engaging and professional presentation.	Produces a moderately detailed report with basic explanations and visuals; presentation is clear but lacks depth or polish.	Report or presentation is poorly structured, lacks clarity, or contains insufficient explanation of results.	

LAB # 07

OPEN ENDED LAB

OBJECTIVE

To apply the foundational machine learning concepts learned in Labs 1–6 to design and implement a small, custom ML project in Python.

The goal is to integrate multiple concepts learned so far into a single, working prototype that performs real-world data analysis or prediction.

PROJECT DESCRIPTION

This is an open-ended lab where you will be told individually to work on the following project ideas and build a working ML solution in Python.

Your project should demonstrate the integration of at least two machine learning models and appropriate data preprocessing, visualization, and evaluation.

PROJECT IDEAS

The project ideas for the Open-Ended Lab will be **provided to students on the day of the lab session**. These ideas will be designed to ensure the integration and practical application of concepts covered in the first six labs. Each project will require students to demonstrate problem-solving, analytical thinking, and implementation of appropriate machine learning techniques.

DELIVERABLES

- All source code for your project, clearly commented with Project name and roll no on top of the notebook.
- Your dataset description and preprocessing steps.
- The ML models used and how they were integrated.
- Model evaluation (metrics, confusion matrix, etc.).
- Code must be uploaded on GitHub and link paste on QOBE portal.